STRIPE PROJECT

4. Data Pipeline

Annexes

4.A. Data Pipeline Tools Rationale

> *Nicolas Pichon --- AIA RNCP 38777 / BC02 / D5 : Data Pipeline Architecture / Annexe A / v11 --- 2026/11/05.*

- 4.A.1. Contexte
- 4.A.2. Capture des changements (_CDC_) : _Debezium_
- 4.A.3. Contrats de données : _Confluent Schema Registry_
- 4.A.4. Traitement de flux : _Apache Flink_
- 4.A.5. Transformation : _dbt_
- 4.A.6. Entrepôt analytique : _Snowflake_
- 4.A.7. Supervision : _Prometheus_ & _Grafana_
- 4.A.8. Qualité des données : _Great Expectations_ + _tests dbt_
- 4.A.9. Synthèse
- Références

4.A.1. Contexte

Cet annexe détaille les justifications des décisions technologiques prises dans [D5].

Technologies hors périmètre

On ne rediscute pas ici la sélection des solutions *Apache Kafka* (bus d'évènements temps réel) et *Apache Airflow* (orchestrateur de processus) - que [R0] demande de considérer comme des premiers choix (cf . exigence technique TR2 [R0]).

On ne reprend pas non plus les outils déjà validés pour les stockages *OLTP* (*PostgreSQL* dans [D2-C]) et *NoSQL* (*MongoDB* dans [D4-A]).

Ce document ne considère que les technologies d'exécution du pipeline non encore résolues : capture, transport, transformation, stockage analytique, supervision.

4.A.2. Capture des changements (*CDC*) : *Debezium*

Pourquoi pas *AWS DMS* ?

AWS DMS (*Database Migration Service*) fait techniquement le travail, mais engage explicitement vers AWS — exactement l'écueil déjà écarté pour l'entrepôt analytique (§4.A.6). Aucun autre choix de ce projet (Kafka, MongoDB, PostgreSQL) n'est spécifique à un fournisseur cloud ; introduire DMS romprait cette neutralité pour un seul maillon de la chaîne.

Ce que *Debezium* apporte spécifiquement

Debezium s'exécute comme connecteur *Kafka Connect* — il s'intègre directement dans l'écosystème déjà retenu (Kafka), sans composant d'infrastructure supplémentaire à opérer séparément. Son support de la réplication logique PostgreSQL est natif et mature — c'est littéralement le mécanisme déjà nommé dans [D1] (*"CDC (Debezium ou réplication logique PostgreSQL)"*), pas un nouvel outil qui s'ajouterait à l'architecture déjà décrite.

4.A.3. Contrats de données : *Confluent Schema Registry*

Pourquoi pas *Apicurio* ?

Apicurio est une alternative *open source* valable, sans lien avec un fournisseur cloud précis — la neutralité n'est donc pas ici le critère décisif, contrairement au CDC ou à l'entrepôt.

Ce que *Confluent Schema Registry* apporte spécifiquement

C'est le registre le plus étroitement intégré à *Debezium* et à *Kafka Connect*, déjà retenus : la compatibilité de schéma (Avro/Protobuf) entre le connecteur CDC et les consommateurs en aval (Flink, Airflow) est directement supportée, sans couche d'interopérabilité à ajouter. Un registre alternatif fonctionnerait, mais réintroduirait une vérification de compatibilité que Confluent Schema Registry résout par construction avec les outils déjà choisis.

4.A.4. Traitement de flux : *Apache Flink*

Pourquoi pas *Kafka Streams* ?

Kafka Streams est une bibliothèque embarquée dans une application JVM, pas un cluster opéré séparément - adapté à un traitement simple, mais moins adapté aux deux usages déjà établis pour ce composant ([D7, §D7.3.3], calcul des features de fraude ; [D5, §D5.9.5], alerting sécurité sur `audit.access`) : deux jobs à état, avec fenêtres glissantes, qui bénéficient d'être opérés sur un cluster partagé plutôt que dupliqués en applications JVM indépendantes.

Pourquoi pas *Spark Streaming* ?

Spark Streaming fonctionne par micro-lots, avec un plancher de latence de l'ordre de la seconde — incompatible avec l'exigence déjà posée ([D7] §D7.3, calcul des features en moins de 1 seconde, elle-même contrainte par le budget de bout en bout de 100 ms). *Flink* traite les événements un par un, sans ce plancher structurel.

Ce que *Flink* apporte spécifiquement

Fenêtres glissantes à faible latence, sémantique *exactly-once*, et surtout la capacité déjà exploitée deux fois dans ce projet ([D5, §D5.9.5]) à maintenir un état par entité sur des topics différents, pour des besoins différents, sur un seul moteur.

4.A.5. Transformation : *dbt*

Pourquoi pas *Spark SQL* ?

Spark SQL est pertinent quand la transformation dépasse ce que SQL exprime nativement (traitement distribué complexe, algorithmes itératifs). Les transformations de ce pipeline — chargement du schéma en étoile, fusion SCD2, calcul des mesures dérivées ([D5] §D5.2) — sont fondamentalement des opérations SQL. Introduire *Spark* reviendrait à opérer un second cluster de calcul pour un besoin que l'entrepôt cible peut déjà traiter lui-même.

Ce que *dbt* apporte spécifiquement

dbt exécute ses transformations **directement dans l'entrepôt** (Snowflake, §4.A.6) — pas de cluster de calcul séparé à maintenir. Ses transformations sont versionnées comme du code, et son cadre de tests intégré alimente directement la stratégie de qualité de données (§4.A.8), sans outil supplémentaire pour les vérifications les plus courantes (unicité, non-nullité, intégrité référentielle).

4.A.6. Entrepôt analytique : *Snowflake*

Pourquoi pas *Redshift* ou *BigQuery* ?

Les deux fonctionnent techniquement pour un schéma en étoile de cette taille. Mais chacun engage explicitement vers un fournisseur cloud (AWS pour Redshift, GCP pour BigQuery) — alors qu'aucun autre choix de la stack (Kafka, MongoDB, PostgreSQL, Flink, Airflow) ne l'est. Ce n'est pas un argument théorique : c'est la même logique qui a déjà écarté AWS DMS en CDC (§4.A.2).

Pourquoi pas *ClickHouse* ?

ClickHouse excelle sur des requêtes analytiques à très fort volume et coût de licence nul, mais son support natif du partage de données gouverné entre organisations distinctes est moins mature — un manque direct pour le besoin ci-dessous.

Ce que *Snowflake* apporte spécifiquement

Trois raisons, déjà détaillées dans [D5, §D5.11] :

- **Neutralité cloud** :
  - La stack technique existante (Kafka, MongoDB, PostgreSQL, Flink, Airflow) est sélectionnée pour éviter tout engagement avec un fournisseur *cloud* précis. Typiquement, *Redshift* engagerait vers *AWS*, *BigQuery* vers *GCP*. On applique ce principe de neutralité vis-à-vis du fournisseur des services *cloud* en sélectionnant *Snowflake*.
- **Secure Data Sharing** :
  - [R0] exige un reporting interne *et* un reporting par marchand. Le reporting est déjà cloisonné par *RLS* ([D3] §D3.5). Le mécanisme *Secure Data Sharing* de *Snowflake* étend ce cloisonnement : chaque marchand reçoit une vue gouvernée de ses propres données sans duplication ni API dédiée à construire.
- **Isolation des charges de consommation**
  - Même avec un chargement par lots entièrement planifié ([D1, §D1.3], [D5, §D5.9.5]), plusieurs usages consomment l'entrepôt en parallèle avec des profils différents (tableau de bord marchand, interactif ; reporting conformité, requêtes lourdes planifiées ; chargement ETL). Les *virtual warehouses* de *Snowflake* isolent chaque charge sur son propre compute, sans contention, sans qu'aucune ne ralentisse les autres.

4.A.7. Supervision : *Prometheus* & *Grafana*

Pourquoi pas *Datadog* ?

Datadog offre une expérience clé en main et des intégrations nombreuses, mais reste un SaaS payant, facturé à l'hôte/à la métrique — un modèle de coût qui croît avec la taille du pipeline. Aucun autre composant de cette stack n'est un SaaS propriétaire équivalent (Kafka, Flink, Airflow, dbt sont tous auto-hébergés ou open source) ; introduire Datadog romprait cette cohérence pour la seule couche de supervision.

Ce que *Prometheus* & *Grafana* apportent spécifiquement

Kafka, Flink et Airflow exposent tous des métriques *Prometheus* nativement ou via des exportateurs bien maintenus — l'intégration ne demande pas de développement supplémentaire. Coût de licence nul, cohérent avec le reste d'une stack majoritairement open source.

4.A.8. Qualité des données : *Great Expectations + tests dbt*

Pourquoi pas *Soda* ?

Soda propose un cadre d'assertions comparable, mais sans le même couplage avec le cadre de tests déjà intégré à *dbt* (§4.A.5) — l'adopter dupliquerait une partie de ce que dbt fournit déjà nativement pour les vérifications les plus courantes.

Ce que *Great Expectations + dbt tests* apportent spécifiquement

Les tests dbt (unicité, non-nullité, intégrité référentielle) couvrent gratuitement une large part du besoin, puisque dbt est déjà l'outil de transformation retenu. *Great Expectations* complète pour les vérifications que dbt n'exprime pas aussi naturellement — contrôles de distribution, règles métier inter-tables — sans dupliquer ce que dbt couvre déjà.

4.A.9. Synthèse

Fonction	Retenu	Écarté	Raison principale
CDC	Debezium	AWS DMS	Neutralité cloud
Contrats de données	Confluent Schema Registry	Apicurio	Intégration native avec Debezium
Traitement de flux	Apache Flink	Kafka Streams, Spark Streaming	Latence, état partagé sur deux usages
Transformation	dbt	Spark SQL	Transformations SQL, pas de cluster séparé
Entrepôt analytique	Snowflake	Redshift, BigQuery, ClickHouse	Neutralité cloud, partage gouverné
Supervision	Prometheus, Grafana	Datadog	Cohérence open source, coût
Qualité des données	Great Expectations, tests dbt	Soda	Couplage avec dbt déjà retenu

Un fil conducteur traverse ce tableau, pas sept décisions indépendantes : à chaque fois qu'un choix engageait vers un fournisseur cloud précis (AWS DMS, Redshift, BigQuery), il a

été écarté au profit d'une alternative neutre — cohérent avec le fait qu'aucun autre composant de cette architecture ne dépend d'un cloud particulier.

Références

Spécifications

- [D0] *Stripe Project Introduction*

Livrables

- [D1] *Comprehensive Data Architecture Diagram*
- [D5] *Data Pipeline Architecture*
- [D7] *Machine Learning Integration Strategy*

Annexes

- [D2-C][OLTP Supporting Notes]
- [D3-A] [OLAP Kimball Analysis]
- [D4-A] [NoSQL Supporting Notes]